



Huerto Vivo

Informe Ejecutivo del Proyecto

Sistema digital de gestión para huerto comunitario urbano

Profesores: Boris Bugueño - Alejandro Paolini

Celso Cereceda - Hernan Cortes - Stefano Morales - Sofia Rivero

Stack: PostgreSQL · Node.js/Express/TypeScript · React/Vite

18 vecinos

comunidad

10 tablas BD

relacionales

9 pantallas

funcionales

1. Contexto y Problema

Un grupo de 18 vecinos gestiona un huerto comunitario en un espacio previamente baldío. Aunque la iniciativa comenzó con entusiasmo, la ausencia de herramientas de coordinación generó conflictos internos, duplicación de tareas y un caos comunicacional insostenible centrado en un grupo de WhatsApp con más de 300 mensajes diarios.

Pain Points identificados

Pain Point	Situación actual	Impacto
Sin trazabilidad de riego	Nadie recuerda quién regó la última vez	Plantas sin riego o riego doble
Caos en planificación espacial	Dos personas sembraron tomates en el mismo espacio	Pérdida de semillas e insumos
Cosechas opacas	Alguien se llevó toda la lechuga sin avisar	Desconfianza y peleas entre vecinos
Infoxicación en WhatsApp	300 mensajes/día donde nada queda registrado	Acuerdos perdidos e incumplidos
Agotamiento del liderazgo	Coordinadora (Doña Carmen) al borde de la renuncia	Riesgo de colapso del proyecto
Sin rendición de cuentas	Imposible distinguir activos de incumplidores	Desmotivación de los comprometidos

2. Objetivo del Sistema

Desarrollar una plataforma digital dual — aplicación móvil + dashboard comunitario — que centralice el registro de siembra, riego y cosecha, transformando la comunicación caótica en datos accionables y públicos para los 18 vecinos.

Preguntas que el sistema responde

Pregunta	Fuente de datos	Pantalla
¿Qué parcelas se regaron hoy y quién es el responsable?	turnos_riego	Riego
¿Cuánto se ha cosechado este mes y cómo se reparte?	cosechas + distribuciones	Cosecha
¿Qué vecinos son los más activos y cuáles incumplen?	actividad_vecinos	Inicio / Ranking
¿Qué se sembró, en qué parcela y cuándo?	siembras + especies	Siembras / Mapa

Requerimientos funcionales prioritarios

ID	Requerimiento	Prioridad
RF1	Registro y marcado de turnos de riego con responsable y hora exacta	Alta
RF2	Gestión de siembras: qué, dónde, cuándo y quién — evita duplicados	Alta
RF3	Reporte de cosechas con distribución equitativa automática	Alta
RF4	Dashboard con KPIs en tiempo real visible para todos los miembros	Alta
RF5	Ranking de vecinos por % de cumplimiento de turnos	Alta
RF6	Rotación automática de turnos — coordinador define patrón del mes	Media
RF7	Recordatorios automáticos al asignar turnos	Media
RF8	Mapa visual interactivo de las 18 parcelas	Media
RF9	Chat comunitario persistente en base de datos	Media

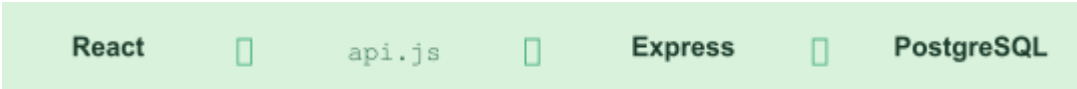
3. Arquitectura del Sistema

Stack tecnológico

Capa	Tecnología	Versión	Justificación
Base de datos	PostgreSQL	18	Relacional — constraints únicos evitan siembras duplicadas
Backend	Node.js + Express + TypeScript	20 LTS	Rápido para peticiones pequeñas, tipado previene errores
Autenticación	JWT + bcrypt	—	Tokens 7 días, hashes seguros, control de roles
Validación	Zod	3.x	Esquemas en todos los endpoints POST/PATCH
Frontend	React + Vite	18/5	Mobile-first, proxy integrado, componentes reutilizables
Estilos	CSS Variables	—	Paleta verde comunitario, sin dependencias externas

Flujo de datos

Cada acción del usuario sigue el mismo recorrido: React (localhost:5173) → api.js (fetch con JWT) → Vite Proxy → Express Backend (localhost:3000) → PostgreSQL (huerto_db). La respuesta regresa en el mismo orden y React actualiza la interfaz sin recargar la página.



Tablas de la base de datos

Tabla	Propósito	Pain point resuelto
vecinos	18 miembros con login (email + bcrypt)	Identificación segura de cada vecino
parcelas	Grilla 6×3 con coordenadas únicas	Evita siembras en el mismo espacio
especies	Catálogo de cultivos disponibles	Estandariza nombres de plantas
siembras	Qué se sembró, dónde, cuándo y quién	RF2: trazabilidad completa del cultivo
turnos_riego	Un responsable por parcela por día	RF1: nadie dice 'no sabía que me tocaba'
cosechas	Registro obligatorio de toda recolección	RF3: elimina extracciones sin avisar
distribuciones_cosecha	Reparto equitativo automático entre vecinos	Transparencia total en el reparto

actividad_vecinos	Métricas mensuales por vecino	RF5: ranking objetivo de compromiso
mensajes_chat	Chat comunitario persistente	Reemplaza los 300 mensajes de WhatsApp
notificaciones	Avisos globales o personales	Recordatorios de turnos y reuniones

4. Funcionalidades del Sistema

Pantallas por tipo de usuario

Pantalla	Acceso	Funcionalidades principales
Login	Pública	Autenticación con email/contraseña. Modo demo sin backend.
Inicio	Todos	KPIs del día, distribución de cosechas, ranking de vecinos (ordenado por % de cumplimiento), próximos turnos.
Siembras	Todos	Lista de siembras activas con barra de progreso calculado. Modal para registrar nueva siembra con selectores de parcela y especie.
Riego	Todos	Turno propio destacado, barra de avance del día, botón 'Marcar como Regado' (guarda en BD), lista completa con estado de cada vecino (■■■).
Cosecha	Todos	Formulario desglosado: cultivo, calidad, kg, fecha, notas. Botón distribuir entre vecinos. Galería de cosechas recientes.
Chat	Todos	Mensajes persistentes en BD. Auto-refresco cada 8 segundos. Envío con Enter.
Mapa	Coordinador	Grilla visual 6×3 interactiva. Colores por estado de riego. Filtros rápidos. Toca para ver detalle completo.
Turnos	Coordinador	Flujo 3 pasos: configurar periodo+intervalo → vista previa → confirmar. Recordatorios automáticos incluidos.
Notificaciones	Todos	Panel con tipos info/riego/cosecha/alerta. Marcar leídas. Coordinador crea avisos globales.

Funcionalidades especiales

Funcionalidad	Descripción	Tecnología
Modo offline	Detecta pérdida de conexión. Acciones guardadas en cola localStorage y sincronizadas automáticamente al reconectar.	<code>navigator.onLine + localStorage</code>
Rotación automática	Genera turnos del mes entero con un clic. Preview antes de confirmar. Intervalo configurable 1-7 días.	<code>POST /api/rotacion/generar</code>
Recordatorios	Al generar rotación, cada turno crea notificación para el vecino. Botón para enviar recordatorios de mañana.	<code>tabla notificaciones</code>
Mapa interactivo	Grilla visual en tiempo real. Colores según estado de riego. Clic en parcela muestra detalle, progreso y responsable.	<code>GET /api/mapa</code>
Roles diferenciados	Miembros ven 5 tabs. Coordinador ve tabs de Mapa y Rotación en lugar de Siembras y Cosecha.	<code>vecino.rol</code> en JWT

5. Resultados y Entregables

Archivos del proyecto

Archivo	Contenido	Tamaño aprox.
huerto_db_completa.sql	Base de datos v2.0 — 10 tablas, 3 vistas, 18 vecinos con bcrypt, datos de prueba completos	~450 líneas
huerto-backend-final.zip	API REST completa — 11 rutas, middleware JWT, validación Zod, transacciones	~800 líneas TS
huerto-frontend-v3.zip	App React — 9 pantallas, modo offline, navegación por roles, api.js centralizado	~2500 líneas JSX
bitacora_huerto_vivo.docx	Documento Word con arquitectura, endpoints, prompts clave y credenciales	7 secciones

Impacto esperado vs. situación inicial

Aspecto	Antes (WhatsApp)	Después (Huerto Vivo)
Trazabilidad de riego	Ninguna — nadie recuerda quién regó	Registro exacto con responsable y hora
Planificación espacial	Conflictos — siembras duplicadas	Constraint único en BD evita duplicados
Cosechas	Opacas — alguien se lleva todo sin avisar	Registro obligatorio + distribución automática
Comunicación	300 mensajes/día sin estructura	Chat persistente + notificaciones tipadas
Compromiso	Subjetivo — peleas por percepción	Ranking objetivo por % de turnos cumplidos
Carga del coordinador	Manual — persiguiendo a cada vecino	Rotación automática + recordatorios

Credenciales para pruebas

Email	Nombre	Rol	Contraseña
carmen@huerto.cl	Carmen Fuentes	Coordinadora	1234
jose@huerto.cl	José Ramírez	Miembro	1234
carlos@huerto.cl	Carlos Flores	Miembro	1234
(cualquier email)	18 vecinos total	Miembro	1234

Este sistema fue construido íntegramente en una sesión colaborativa usando Claude (Anthropic) como asistente de desarrollo, con iteraciones progresivas desde el análisis de requerimientos hasta la implementación completa del stack.